Eventual Coherence: Preserving Causal Validity in Replicated Systems

Solving the Problem of Divergent Intent in Shared State Networks

Forest Mars
mars@mlops.nyc

Abstract

The delete-update conflict, where one device deletes a resource while another concurrently updates it, is an underdetermined challenge in multi-device synchronization. Existing approaches treat it as a synchronization problem rather than a semantics problem and so sacrifice at least one of three properties: availability through coordination, causal validity through last-write-wins semantics, or storage boundedness through conflict-free replicated data types. We argue this tradeoff arises from a category error: treating topology operations and content operations as equivalent and subject to uniform reconciliation semantics. Eventual coherence is the correctness criterion we introduce to resolve it: each replica is evaluated against its own causal history rather than required to agree with the others. Control plane operations establish resource topology and reconcile at synchronization boundaries; data plane operations modify resource content and propagate in near-real time. Reconciliation is governed by two principles: local deletion finality (a device that has deleted a resource permanently discards subsequent updates to it) and server-level upsert semantics (a server receiving an update for a deleted resource restores it, so devices with active causal participation keep receiving updates). Neither device is forced to agree with the other; each retains the outcome its own causal history entitles it to, and tombstone storage is bounded by outstanding causal divergence rather than accumulated deletion history. The protocol rests entirely on Lamport clock ordering, requiring no vector clocks, no operational transformation, and no physical clock synchronization, and is implementable over standard HTTP. We prove the protocol guarantees per-device causal validity, convergence to causally valid states, update preservation, and local deletion stability, and that inter-replica agreement and per-device causal validity are formally orthogonal criteria, making eventual coherence and eventual consistency incomparable rather than points on the same hierarchy. We validate the protocol against Polyglot, a proof-of-concept multi-device conversation management system.

1. Introduction

Distributed systems that maintain replicated state across multiple devices face a persistent challenge: how to reconcile operations when devices independently modify shared resources [2, 1, 3]. This problem becomes particularly acute when one device deletes a resource while another concurrently updates it. Consider a conversation management system where Device A deletes a conversation, while Device B, operating without knowledge of this deletion, continues adding messages to it. When these devices synchronize, the system must decide: should the deletion propagate, discarding Device B's work, or should the update propagate, implicitly reversing the deletion?

Neither outcome is correct. Device A made a deliberate choice to delete. Device B made a deliberate choice to continue working. A system that erases either choice in the name of convergence has not achieved correctness. It has imposed an arbitrary resolution on a situation that does not require one.

Traditional approaches to this problem fall into three categories, each sacrificing at least one property the other two preserve. Coordination-based systems achieve consistency by requiring devices to obtain locks [3] or reach consensus [4, 5, 6] before performing operations, but this approach sacrifices availability when devices cannot communicate [7], precisely the conditions under which multi-device systems are most valuable [8, 9]. Last-write-wins (LWW) strategies avoid coordination by imposing total ordering based on timestamps [10, 11], but this discards causal validity: a deletion performed after an update may be overridden by that update due to clock skew, [1] or an update

performed after a deletion may be discarded when the deletion carries a later timestamp, losing work performed in good faith. Conflict-free replicated data types [3, 14] preserve all operations through carefully designed merge semantics, but make true deletion difficult, requiring either tombstones that accumulate without bound [15, 16] or monotonic state growth [14] that prevents genuine removal.

All of these approaches introduce substantial implementation complexity, forcing application developers to reason about specialized data structures, merge semantics, and long-term state management concerns that arise primarily from the synchronization mechanism itself rather than the underlying application domain.

We observe that this problem arises from a category error that runs through all three approaches: treating all operations as equivalent writes subject to uniform reconciliation semantics. Resource creation and deletion establish the topology of the system, what resources exist at all. Resource updates modify the content of existing resources without changing topology. These are different kinds of operation having different causal properties [14, 18. 14], different timing requirements, and different correct reconciliation behaviors. Handling them uniformly produces systems that are simultaneously harder to build and less faithful to user intent.

This paper introduces eventual coherence, a synchronization model built on the explicit categorical separation of these operation classes. Control plane operations (CREATE, DELETE) establish resource topology. They are reconciled at synchronization boundaries, defined points at which devices exchange control plane state with the server. Data plane operations (UPDATE) modify resource content. They propagate in near-real-time throughout normal device operation. This separation is not a performance optimization. It reflects a genuine difference in the nature of the operations and their role in establishing system invariants.

Reconciliation in eventual coherence is governed by two principles. The first is local deletion finality: a device that has deleted a resource permanently ignores all subsequent data plane updates targeting that resource. The deletion is a terminal state for that device's relationship with the resource. The second is server-level upsert semantics: when a server receives a data plane update for a resource currently marked as deleted, it restores the resource. This ensures that devices with active causal participation in the resource continue to receive updates from other participants.

These two principles together resolve the delete-update conflict. The deleting device maintains its deletion. Devices that were actively using the resource continue to do so. Third-party devices receive the most recent updates. No coordination is required. No tombstones accumulate. No causal validity is violated for any participating device.

The formal contribution of this paper is the introduction of eventual coherence, a correctness criterion that operates incommensurable with the classical distributed consistency hierarchy: that hierarchy measures degrees of cross-replica agreement, whereas eventual coherence evaluates per-replica causal validity, a property the agreement hierarchy neither expresses nor implies. The traditional hierarchy is fundamentally an agreement hierarchy; it assumes that a distributed system's proximity to correctness is proportional to its total and fine-grained consensus. This assumption holds for systems modeling a single source of objective reality, such as financial ledgers. It fails catastrophically when replicas are extensions of independent human actors. A system that erases an explicit local action to force cross-replica agreement has not achieved correctness; it has achieved conformity by destroying information.

Eventual coherence rejects the assumption that agreement is the sole proxy for correctness, treating inter-replica agreement and historical validity as independent, orthogonal dimensions of a broader correctness space. This criterion is not universally appropriate. Systems where replicas must agree on a single, shared authoritative state, such as financial ledgers or inventory management, rightly prioritize consistency. Eventual coherence targets systems where devices represent independent human agents operating in distinct causal contexts, and where forcing convergence means erasing legitimate work.

This approach provides several advantages over existing methods. Unlike coordination-based systems [19, 20], it operates correctly without requiring devices to communicate before performing operations. Unlike last-write-wins [10, 24], it preserves causal validity: updates performed in the context of a resource's existence are never discarded due to timestamp ordering, and deletions are never overridden by remote data plane operations on the deleting device. Unlike conflict-free replicated data types [10, 24], it requires no operational transformation, supports true deletion, and does not require unbounded storage growth. Most significantly, it achieves these properties without vector clocks, without distributed consensus, and with a protocol simple enough to implement over standard HTTP [25].

We make the following contributions:

- A formal definition of eventual coherence as a correctness criterion grounded in Lamport clock ordering, establishing its precise relationship to existing consistency models and proving it is incomparable to eventual consistency rather than merely weaker.
- A synchronization protocol that achieves eventual coherence through two principles, local deletion finality and server-level upsert semantics, requiring no physical clock synchronization, no vector clocks, and no operational transformation.

- Proofs that the protocol guarantees per-device causal validity, local deletion finality, and convergence to causally valid states within the bounds of Lamport clock ordering.
- A proof-of-concept implementation in Polyglot, a multi-device conversation management system, demonstrating practical realizability using standard web technologies, with a full reference implementation in progress

The remainder of this paper is organized as follows: Section 2 presents the system model and formal definitions. Section 3 defines the eventual coherence protocol. Section 4 proves correctness properties. Section 5 is the conclusion. Appendix I provides the protocol algorithmic specifications. Appendix II is formal verification and extended correctness proofs. Appendix III surveys related work. Appendix IV describes a proof-of-concept implementation. Appendix V evaluates performance.

2. System Model and Definitions

We model a distributed system consisting of a finite set of continuously connected devices $\mathscr{D} = \{d_1, d_2, ..., d_n\}$ that replicate a shared set of resources $\mathbb{R}$. Each resource r in $\mathbb{R}$ has a unique identifier and represents a container for mutable content (in our implementation, a conversation consisting of messages). Devices perform operations on resources and synchronize with a server S through a disciplined message passing model described below. This model follows the client-server architecture common in mobile and web applications [26, 27] rather than the peer-to-peer model of some replicated systems [2, 24].

A central premise of this model is that continuous connectivity does not imply, nor require, uniform synchronization of all operation classes. Devices deliberately propagate different classes of operations at different frequencies. This is not a concession to network unreliability. It is an architectural principle reflecting the categorical difference between the operations themselves.

2.1 Synchronization Boundaries

We introduce synchronization boundaries as a first-class concept in the model. A synchronization boundary is a point at which a device performs control plane reconciliation with S. The definition of a synchronization boundary is left to the implementor: it may correspond to application launch, page load, explicit user action (such as initialization), or any other meaningful transition in device state. Between synchronization boundaries, a device operates within a local synchronization epoch, during which it processes data plane operations in near-real-time but does not pull control plane state from S.

> **Definition 1 (Synchronization Epoch)**
> *A synchronization epoch for device d_i is the interval between two consecutive synchronization boundaries. Within a synchronization epoch, di processes data plane operations in near-real-time and accumulates control plane operations for reconciliation at the next synchronization boundary.*

2.2 Lamport Clocks

All causal ordering in the eventual coherence model is established via Lamport clocks [1]. No assumptions are made about physical clock synchronization across devices.

> **Definition 2 (Lamport Clock)**
> *Each device d_i maintains a local Lamport counter L_i, initialized to 0, obeying the following rules:*
> *• On each local operation: $L_i := L_i + 1$*
> *• On sending a message: current value of L_i is included*
> *• On receiving a message carrying Lamport value L_{msg}: $L_i := max(L_i, L_{msg}) + 1$*
> *Server S maintains a global Lamport counter L_s updated by the same rule on every message received.*

> **Definition 3 (Lamport Timestamp)**
> *Each operation o carries a Lamport timestamp L(o) = (L_i, d_i) where d_i is the issuing device, used for deterministic tie-breaking. The total order on Lamport timestamps is: (L_1, d_1) precedes (L_2, D_2) if $L_1 < L_2$, or $L_1 = L_2$ and d_1 precedes d2 lexicographically. This total order is consistent with the happens-before relation*

Physical timestamps may be maintained for human-readable display purposes but are not load-bearing in the protocol. All ordering decisions use Lamport timestamps exclusively.

2.3 Operations

We define two categorically distinct classes of operations. This distinction is the architectural foundation of eventual coherence and reflects a genuine difference in the nature of the operations, not merely a difference in propagation timing.

Definition 4 (Control Plane Operations)

A control plane operation is a function that modifies the topology of the resource set $\mathbb{R}$*. Control plane operations are pushed to* $\mathbb{S}$ *immediately upon execution and pulled from* $\mathbb{S}$ *only at synchronization boundaries. We consider two control plane operations:*

• *CREATE(r): Add resource* r *to* $\mathbb{R}$*, with* $r \notin \mathbb{R}$ *before the operation.*

• *DELETE(r): Mark resource* $r \in \mathbb{R}$ *as deleted on the issuing device.*

Definition 5 (Data Plane Operations)

A data plane operation is a function that modifies the content of an existing resource without changing resource topology. Data plane operations propagate to $\mathbb{S}$ *in near-real-time throughout a synchronization epoch.*

• *UPDATE(r, delta): Apply modification delta to resource* $r \in \mathbb{R}$.

Each operation o carries a Lamport timestamp L(o) as defined in Definition 3, a device identifier o.d, and a resource identifier o.r. This operational model differs structurally from state-based CRDTs [28, 29], which transmit entire states over an anti-entropy layer, and operation-based CRDTs [14], which require causal broadcast infrastructure [31, 44] to guarantee that operations are never processed before their (causal) predecessors, though pure operation-based variants [30] relax this requirement. Eventual coherence requires neither complete state transmission nor causal delivery guarantees. Unlike pure operation-based which achieve delivery-order independence by design of the operation payload itself, eventual coherence achieves it through comparison at the point of application. compareLamport (Section 3.4) re-derives the correct outcome from any two operations' timestamps regardless of arrival order, requiring no special payload structure and no delivery-order guarantees from the transport layer.

2.4 Causal History

Definition 6 (Local History)

The local history H_i *of device* d_i *is the sequence of operations performed by* d_i *totally ordered by Lamport timestamp L(o).*

We write $\tau_o \succ \tau_{\text{delete}}$ *to denote this relation:*

Definition 7 (Causal Participation)

Device d_i *causally participates in resource* r *with respect to a deletion record* $\tau_{\text{delete}} = (l_d, d_d)$ *if there exists an operation* $o \in H_i$ *such that* $o.r = r$ *and the Lamport timestamp of* o*, written* $\tau_0 = (l_d, d_d)$*, strictly dominates* τ_{delete} *lexicographically:*
$l_o > l_d, or (l_o = l_d \ and \ d_o > d_d)$.

Causal participation determines how a device responds to control plane operations at synchronization boundaries. A device that has modified a resource with a Lamport timestamp that strictly dominates the deletion record has a causal stake in the resource's continued existence. This notion tracks participation rather than complete causal history, which is both sufficient for our purposes and significantly simpler to implement than vector clock approaches [32, 33, 34].

2.5 System Assumptions

We assume an asynchronous network model [41, 44] where message delays are unbounded but finite. Devices may crash and recover [36]. We assume crash-recovery failures only, with no Byzantine behavior [37]. $\mathbb{S}$ is assumed available and crash-recoverable with persistent storage [4, 38].

$\mathbb{S}$ serves two functions. First, it maintains persistent current state for all resources, enabling devices to synchronize data plane state at any time. Second, it maintains a history of control plane events sufficient for devices to reconcile at synchronization boundaries. $\mathbb{S}$ broadcasts data plane updates to all connected devices in near-real-time. $\mathbb{S}$ does not maintain per-device state and has no knowledge of which devices have performed which local operations. $\mathbb{S}$ is not a source of truth for individual device state. It is a shared communication and persistence layer.

Devices maintain local replicas in persistent storage [8] and perform all operations locally without coordination [49]. Data plane operations are pushed to $\mathbb{S}$ immediately. Control plane operations are pushed to $\mathbb{S}$ immediately but pulled from S only at synchronization boundaries. We make no assumption about operation ordering across devices, as in the asynchronous model of distributed computing [44, 41]. This model is weaker than systems requiring quorum reads and writes [10] but stronger than systems with no server coordination [42, 48]. It reflects the practical reality of modern mobile and web applications where a shared server provides a convergence point without requiring real-time coordination for all operation classes [26, 49].

A note on scope: The model assumes data plane operations on a given resource originate from a single agent operating across multiple devices. Concurrent authorship, with distinct authors independently targeting the same resource within the same synchronization epoch, is outside the present model's scope. (Under concurrent authorship, Lamport-ordered last-write-wins at the resource level discards one author's operations, violating Property 6.) Multi-agent data plane semantics are left to a subsequent model.

2.6 Consistency Properties and Orthogonality

Classical distributed consistency models operate on a foundational assumption: that a system's proximity to

correctness is directly proportional to its inter-replica agreement. We argue this is a framework error when replicas serve as distinct causal proxies for intent, whether human or agentic. To evaluate such systems, we decouple correctness into two orthogonal dimensions: the Agreement Axis and the Coherence Axis. Let $\sigma(d_i, t)$ represent the materialized state of a resource set on device d_i at logical time t.

Definition 8 (The Agreement Axis).

A system satisfies the Agreement Axis if for any two devices:

$$d_i, d_j \in \mathcal{D},$$

their materialized states asymptotically converge under a terminal execution trace:

$$\lim_{t\to\infty} \sigma(d_i, t) = \sigma(d_j, t)$$

The Agreement Axis as defined here corresponds to the standard formulation of eventual consistency [10]: absent continued updates, all replicas converge to an identical state.

Definition 9 (Coherence / Historical Validity)

A system satisfies the Coherence Axis for device d_i if, for every locally committed operation $o \in H_i$ that is causally admissible, the operational mutations induced by o are deterministically preserved in $\sigma(d_i, t)$ regardless of any concurrent remote operations $o' \in H_j (j \neq i)$. An operation o targeting resource r is causally admissible if no deletion record exists for r, or if τ_o strictly dominates the deletion record τ_{delete} for r lexicographically. Operations for which $\tau_0 \succ \tau_{delete}$ holds are seen to lie outside the causal horizon and such operations may be superseded by the deletion; such supersession is not a coherence violation.

Definition 10 (Eventual Coherence)

A system provides eventual coherence if, for any resource r, when all operations on r cease and all pending synchronization boundaries have been crossed, each device's replica of r converges to a state that is causally valid given that device's local history, as ordered by the Lamport happens-before relation.

Proposition 1 (Orthogonality of Frameworks).

The Agreement Axis and the Coherence Axis are mutually orthogonal correctness criteria. A system may satisfy the Agreement Axis while violating the Coherence Axis on one or more devices, or conversely, satisfy the Coherence Axis on all devices while maintaining permanent state divergence:

$$\lim_{t\to\infty} \sigma(d_i, t) \neq \sigma(d_j, t)$$

2.7 Problem Statement (The Reconciliation Problem).

Given the system model above, we now formulate the reconciliation problem addressed by eventual coherence. The objective is not simply to achieve convergence, but to preserve causal validity while allowing independent devices to operate continuously without coordination.

Design a synchronization protocol such that:

1. Devices can perform data plane operations continuously within a synchronization epoch without coordination.

2. Control plane operations propagate to S immediately upon execution and to all devices at synchronization boundaries.

3. Data plane operations synchronize in near-real-time throughout each synchronization epoch.

4. After a synchronization boundary is crossed, each device's state is causally valid with respect to the Lamport happens-before relation.

5. A device that has deleted a resource is never required to restore it due to remote data plane ops.

6. No device loses data plane operations it performed within a synchronization epoch, provided those operations do not conflict with concurrent data plane operations from a distinct author targeting the same resource.

7. Deletion metadata does not accumulate without bound; once all devices have crossed synchronization boundary since a deletion to make their participation decision, corresponding metadata is eligible for garbage collection. .

Traditional approaches violate at least one of these requirements. Coordination-based systems [4, 20] violate (1) due to the CAP theorem [7, 43]. Last-write-wins systems [10, 50] violate either (5) or (6) depending on timestamp ordering. CRDT systems [14, 15] make true deletion difficult, violating (7) through tombstone accumulation [16, 14] or monotonic state growth [14]. Systems using operational transformation [45, 46] achieve (6) but at significant implementation complexity [47]. Our protocol satisfies all seven properties through explicit plane separation, Lamport clock ordering, local deletion finality, and server-level upsert semantics.

3. The Protocol

The eventual coherence protocol consists of four components: local operation execution, real-time data plane synchronization, real-time control plane push, and control plane reconciliation at synchronization boundaries. We present the formal invariants and operational mechanics of each component in turn. The concrete pseudo-code specifications and reference typing structures for all client and server execution blocks are detailed in Appendices I/IV.

3.1 Local Operation Execution

All operations execute locally without network coordination, adhering to the optimistic replication paradigm [48, 49]. When a device d_i initiates an operation o targeting a resource r, it immediately applies the state changes to its local replica, advances its tracking variables, and records o within its local history H_i.

Control plane operations govern resource topology (creation and deletion). The execution of a local deletion action does not physically destroy or purge the underlying data structure from local storage. Instead, it marks the resource as locally inactive and establishes an immutable logical timeline marker. The specific structural fields committed during local topology allocation are defined algorithmically in Appendix I.1.

For data plane operations (content mutations), the client proxy evaluates the state relation before application. If a device has locally marked a resource as deleted, all subsequent data plane operations targeting that resource are silently discarded. The deletion acts as a terminal state for that specific device's active causal relationship with the resource. This rule formalizes the boundary condition for Local Deletion Finality (defined explicitly in Section 3.5)..

3.2 Real-Time Data Plane Synchronization

When a device successfully executes a local UPDATE operation, it immediately dispatches the content delta and its associated logical tracking primitives to the server S. Upon receipt, S advances its global tracking counters and resolves concurrent content updates using Last-Write-Wins (LWW) semantics ordered by strict logical dominance.

To handle concurrent interactions across decoupled planes, the server implements a stateless propagation upsert mechanism, preserving deletion metadata without acting as a consistency authority. S does not resolve correctness it relays state transitions for later device-side reconciliation.When S receives a data plane update for a resource currently marked as structurally deleted in its own storage layer, it restores the active visibility of the resource and broadcasts the mutation to all currently connected proxies. Crucially, S preserves the historical deletion record; it is never overwritten or cleared by data plane operations.

Third-party devices that possess an active relationship with the resource receive this broadcast and update their local states via LWW semantics. Reconciling devices that have already executed a local deletion also receive the broadcast but immediately drop it from execution.

This upsert mechanism presupposes that the receiving proxy possesses prior topological knowledge of the target resource. If a device receives a real-time data plane broadcast for a resource ID that does not exist within its local storage layer (neither as an active resource nor as a retained deletion marker), it is prohibited from materializing the content. Allowing a data plane update to spontaneously originate a local resource would collapse the structural boundary between topology and content. Instead, the device treats the arrival of the broadcast as affirmative evidence of an un-pulled upstream state and signals that a synchronization boundary is available. The formal interface handlers governing non-originating broadcasts are provided in Appendix I.2.

3.3 Real-Time Control Plane Push

Control plane operations are pushed to S immediately upon local execution. This establishes the push side of the asymmetric control plane propagation model: proxies push topology events in real-time but pull remote changes exclusively at synchronization boundaries.

When S processes an incoming deletion message, it updates its persistent metadata state and tracks the deletion horizon. Crucially, S does not broadcast deletions (or any control plane operations) to connected devices in real-time; propagation is deferred until the target proxies independently cross a synchronization boundary.

During the temporal window between a local deletion event on one proxy and a boundary crossing on another, the server may oscillate between deleted and restored states as real-time control plane pushes and un-synchronized data plane updates alternate. This oscillation is a natural property of an optimistically replicated system where the server functions as a non-evaluative relay rather than a centralized consensus authority; consistency resolves deterministically once all active proxies cross their respective synchronization boundaries. The server-side message reception routines are structurally mapped in Appendix I.3.

3.4 Control Plane Reconciliation at Synchronization Boundaries

When a device d_i initiates a synchronization boundary event, it executes a multi-step reconciliation cycle. It pulls the current server metadata and state ledgers, then evaluates its local state map against the incoming data and control plane horizons. The reconciliation logic evaluates three distinct boundary conditions:

Case 1 (Local Deletion Finality): Evaluated unconditionally prior to all other states. If the local resource

is marked as deleted, the proxy skips all subsequent content merges, re-informs S of its deletion horizon to supersede any active upserts, and terminates evaluation for that resource.

Case 2 (Causal Participation Detection): If a remote deletion record matches a locally active resource, the engine evaluates whether the local device maintained active causal participation past the deletion horizon. If the local update does not strictly dominate the deletion timestamp, the local resource is dropped and the deletion horizon is tracked. Concurrent operations at identical logical counter values default to deleted, guaranteeing deterministic convergence without cross-proxy coordination.

Case 3 (Standard LWW Content Merge): In the absence of a deletion record on either the local proxy or the server, state mutations are merged via standard Lamport-ordered Last-Write-Wins.

The detailed sequential implementation of this evaluation engine and its tie-breaking dominance routines are formalized in Appendix I.4.

3.5 Correctness Invariants

The execution model is bound by seven core invariants:

> **Invariant 1 (Local Causal Consistency)**
> *Within a single synchronization epoch on device d_i, all operations respect the happens-before relation [1].*
>
> *If $o_1 \rightarrow o_2$ on d_i, then $L(o_1) < L(o_2)$ in H_i. This is guaranteed by the Lamport clock rules: each operation increments L_i before executing.*

> **Invariant 2 (Control Plane Idempotence)**
> *Applying the same control plane operation multiple times has the same effect as applying it once. CREATE is idempotent because creating an already-existing resource is a no-op.*
>
> *DELETE is idempotent because marking a deleted resource as deleted again changes nothing. This ensures that network failures and retries do not violate correctness [50, 51].*

Local deletion finality is a device-local evaluation rule, not a system-wide state mutation.

> **Invariant 3 (Data Plane Convergence)**
> *If all devices cease data plane operations and all pending data plane synchronizations complete, all devices that have not deleted a resource converge to the same content for that resource. This follows from Lamport-ordered last-write-wins: the version with the highest Lamport timestamp eventually propagates to all non-deleted replicas.*

> **Invariant 4 (Causal Participation Preservation)**
> *If a device d_i executes a data plane update* u *on a resource* R *within its local history H_i, the operational intent of* u *is globally preserved across subsequent synchronization boundaries, provided that the device's local Lamport counter has been advanced past the deletion counter l_d associated with any remote control plane deletion of* R *prior to executing* u.

> **Invariant 5 (Local Deletion Finality)**
> *If device d_i has marked resource* r *as deleted, all subsequent data plane operations targeting* r *on d_i are silently discarded. Local deletion is a terminal state for a device's relationship with a resource. It cannot be overridden by remote data plane operations. This invariant applies both during normal operation (Section 3.1) and during control plane reconciliation (Section 3.4, Case 1). To sustain this invariant, a device that deletes* r *must retain a minimal local deletion record, i.e. {*r.id, r.deletedAtLamport*}. This record must not be removed by any data plane operation. Its presence is what allows the receive handler (Section 3.2) to distinguish a deleted resource from a resource the device has never seen. Without it, both cases produce a null result from* local_storage.get(r.id)*, and the two invariants become unenforceable simultaneously.*

> **Invariant 6 (Data Plane Non-Origination)**
> *A device that receives a real-time data plane broadcast for a resource* r *for which it holds no local record, neither active nor deleted, does not apply the broadcast content to local storage. The device's Lamport counter advances per Definition 2's unconditional receive rule. The broadcast's arrival constitutes affirmative evidence that a control plane CREATE for* r *exists at* S *which the device has not yet pulled; the device therefore signals that a synchronization boundary is available. Topological acquisition of* r*, including its content as of the broadcast, is deferred to the device's next synchronization boundary, where Case 3 of the reconciliation algorithm merges the server's current state for* r *via standard Lamport-ordered last-write-wins. The invariant prohibits content materialization from data plane events; it does not prohibit using such events as boundary availability signals. Compliant implementations may surface this signal as a user-visible notification or use it to trigger an immediate boundary crossing.*

The retention requirement of Invariant 5 is what makes a null result from local_storage.get(r.id) semantically unambiguous. With it in force, a device that returns null for some resource id is guaranteed to have never acquired that resource; deleted resources remain present in local storage with is_deleted=true. This guarantee is load-bearing for the

case where a device may receive a real-time data plane broadcast for a resource it has no topological knowledge of, e.g. when another device creates and updates a resource within a single synchronization epoch before the receiving device has crossed a boundary. The receive handler must distinguish this case from a locally deleted resource and respond differently to each, which distinction is only available if Invariant 5's retention requirement holds.

A device may receive a real-time data plane broadcast for a resource it has no topological knowledge of, for instance when another device both creates and updates a resource within a single synchronization epoch before the receiving device has crossed a boundary, which case is governed by Invariant 6.

To guarantee that the coordination environment can accurately preserve execution history for evaluation by reconciling devices, the server must enforce metadata durability for deletion records.

> **Invariant 7 (Deletion Record Timestamp Preservation)** *Let DR be the server-side deletion record for a deleted resource R, containing the definitive logical deletion timestamp $\tau_{deletedAt}$.*
>
> *From the moment of initial control plane registration until final metadata garbage collection, $\tau_{deletedAt}$ must be strictly immutable. The coordination server is explicitly prohibited from overwriting, shifting, or truncating $\tau_{deletedAt}$ in response to incoming data plane updates*

3.6 Garbage Collection

Client-side deletion records are subject to a symmetric purge condition. A device may remove its local deletion record from its underlying data store when it crosses a synchronization boundary and the target resource ID is absent from the server's active resource list. Absence at a synchronization boundary serves as the observable signal that S has completed server-side garbage collection for the resource, which guarantees that no future data-plane updates or broadcasts for that resource ID will be issued.

The deletion record's disambiguating function is exhausted at that point. A device must not purge a local deletion record on any other basis; elapsed time, storage pressure, or application-layer signals are not safe purge triggers, as none of them establish that a resource ID will never reappear in a future broadcast.

This formulation requires S to track synchronization boundary crossings per device, which is minimal per-device state. V2 of the protocol eliminates this requirement by having devices explicitly signal completion of synchronization boundary processing, removing server-side tracking entirely. Device lifecycle presents an open problem for garbage collection. Devices that are permanently decommissioned or lost will block garbage collection indefinitely if S awaits their synchronization boundary crossing. Characterizing the interaction between device lifecycle and garbage collection is left to future work.

3.7 Complexity Analysis

Communication Complexity: Data plane updates require O(1) messages per operation (device to server, server broadcast to connected devices). Control plane push requires O(1) messages per control plane operation. Control plane reconciliation at a synchronization boundary requires O(|R|) messages, where |R| is the total number of resources. Only logical timestamps and deletion records are exchanged during reconciliation, not complete resource state blocks.

Storage Complexity: Each device stores O($|R_{active}|$) resources, where R_{active} is the subset of R not deleted from that device's perspective. S stores O(|R|) resources plus O($|R_{deleted}|$) deletion records pending garbage collection. Unlike operation-based CRDTs [3, 30] that must store complete operation logs [30], eventual coherence requires only current state plus deletion records.

Time Complexity: Local operations complete in O(1) time. Reconciliation at a synchronization boundary requires O(|R|) comparisons. This is optimal: the device must examine each resource entry to determine its participation status.

4. Correctness Proofs

We establish the safety and liveness properties of the eventual coherence protocol by verifying its behavior under arbitrary, concurrent interleavings of control plane and data plane operations. Let $\mathcal{D}$ be a set of independent client devices, and let S be the central coordination server. The global state history is modeled as a sequence of discrete epochs bounded by causal boundary crossings.

4.1 Per-Device Causal Validity

We begin by establishing the primary correctness property of eventual coherence. We do not claim that the protocol satisfies classical causal consistency, which Ahamad et al. [18] define as requiring all devices to observe causally related operations in the same order globally. The protocol explicitly permits permanent divergence between devices with incompatible divergent causal histories. Every independent proxy $d_i \in \mathcal{D}$ must maintain a locally consistent execution trajectory where operations conform strictly to the partial order defined by the happens-before relation ($\rightarrow$).

> **Theorem 1 (Per-Device Causal Validity)** *For any operations o_1, o_2 executed sequentially within a single epoch ond_i, if $o_1 \rightarrow o_2$, then their logical timestamps satisfy $\tau(o_1) < \tau(o_2)$.*

Proof Sketch. This property follows directly from the monotonic advancement of the local Lamport clock layer. Because every local invocation (CREATE, DELETE, UPDATE) forces an isolated scalar increment $L_i \leftarrow L_i + 1$ prior to persistence, logical time is guaranteed to map strictly to causal sequence. The complete proof is detailed in Appendix II.1.

4.2 Convergence to Causally Valid States

> **Theorem 2 (Convergence)**
> *When all operations cease and all pending synchronization boundaries have been crossed, each device converges to a state that is causally valid given its local history.*

In the absence of a centralized coordination authority, concurrent updates must resolve deterministically across all active replicas once network messages settle.

Proof Sketch. Resolution relies on the total ordering of the tuple dominance operator ($\succ$). Because any two matching clock counters are broken deterministically by the strict lexicographical ordering of unique device identifiers ($d_a >_{lex} d_b$), every proxy evaluates an identical winning path under Last-Write-Wins (LWW) semantics. The complete proof is detailed in Appendix II.2.

4.3 Update Preservation

Causal intent must be protected from premature erasure by asynchronous, concurrent control plane operations. The complete proof is detailed in Appendix II.3.

> **Theorem 3 (Update Preservation)**
> *Let τ_o be the logical timestamp of a data plane operation o committed locally on device d_i. Upon crossing a subsequent synchronization boundary, o is deterministically preserved in d_i's materialized state unless there exists a control plane deletion record τ_{delete} such that $\tau_o \not\succ \tau_{delete}$.*

Proof Sketch. Case 2 of the reconciliation algorithm evaluates the participation predicate using full tuple ordering. If $\tau_{local} \succ \tau_{del}$, the engine asserts that d_i operated with prior causal knowledge of the deletion, forcing a server-side upsert that preserves the mutation. The complete proof is detailed in Appendix II.3.

4.4 Local Deletion Stability

Once a proxy explicitly withdraws participation from a resource, its structural isolation must be permanently preserved against downstream real-time network activity.

> **Theorem 4 (Local Deletion Stability)**
> *If device d_i has marked resource $\mathfrak{r}$ as deleted, then for all subsequent synchronization boundary crossings, d_i maintains $\mathfrak{r}$ as deleted in its local state.*

Proof Sketch. This stability condition fulfills Invariant 5. Both the real-time broadcast receive handler and Case 1 of the boundary reconciliation engine execute an immediate short-circuit check against the retained local deletion marker, dropping inbound content frames unconditionally. The complete proof is detailed in Appendix II.4.

> **Corollary 1**
> *Local deletion finality is permanent absent an explicit control plane CREATE operation issued by an administrative authority with a Lamport timestamp higher than the existing deletion record. Such an operation, received by d_i at a synchronization boundary, would appear as a new resource in S's state without a deletion record post-dating the CREATE, allowing d_i to acquire the resource through Case 3 reconciliation. The issuance and authorization of such operations is outside the scope of the synchronization protocol.*

The synchronization protocol supports, but does not implement, a second administrative operation: a full resynchronization that issues a CREATE for every resource currently retained by any device but deleted on the requesting device, restoring the requesting device's complete view in one pass. This operation enters the protocol as a batch of explicit control plane CREATE events with Lamport timestamps exceeding the corresponding deletion records; the protocol's only obligation is to preserve the state needed to construct them, which Invariant 7 (Deletion Record Timestamp Preservation, Section 3.6) guarantees. The issuance, authorization, and sequencing of both administrative operations are application-layer concerns outside the scope of the synchronization protocol.

4.5 Proof of the Orthogonality Proposition

We now prove the Orthogonality Proposition formulated in Section 2.6. To demonstrate that the Agreement Axis and the Coherence Axis are mutually orthogonal, we establish two distinct conditions: (1) a system can achieve perfect inter-replica agreement while comprehensively violating historical validity, and (2) a system can guarantee perfect historical validity while permitting permanent state divergence. Orthogonality is established by constructing two systems with identical agreement behavior but differing coherence outcomes, and vice versa.

> **Lemma 1 (Agreement Without Coherence)**
> *Let $\mathbb{A}$ be a synchronization system whose correctness criterion requires global convergence to a single terminal state. Consider concurrent operations DELETE($\mathfrak{r}$) and UPDATE($\mathfrak{r}$, δ). Any deterministic reconciliation rule in $\mathbb{A}$ that satisfies the Agreement Axis by producing a unique, unified terminal state ($\sigma(d_1) = \sigma(d_2)$) must discard the structural effect of at least one committed operation. If the system converges to a deleted state, the committed update is erased; if it converges to an active state, the terminal deletion is reversed. In either execution path, system $\mathbb{A}$ violates the Coherence Axis for at least one device, establishing that satisfaction of the Agreement Axis does not imply satisfaction of the Coherence Axis.*

Lemma 2 (Coherence Without Agreement)
Let $\mathbb{B}$ *be a synchronization system operating under the eventual coherence protocol. In the identical concurrent scenario:* d_1 *issues DELETE(r) and* d_2 *issues UPDATE(r, δ). Suppose* d_2*'s local Lamport counter has been advanced past τ_delete = ⟨ld, dd⟩ prior to issuing UPDATE(r, δ). Upon crossing a synchronization boundary, under Case 1 of the reconciliation algorithm,* d_1 *unconditionally maintains its local deletion state (σ(d1) = ∅). Concurrently, under Case 2, because our setup assumption guarantees that* $\tau_{local} \succ \tau_{delete}$*,* d_2 *retains the resource and materializes its operational update (σ(*d_2*) = {r ↦ δ}). As t → ∞, the states remain permanently diverged: σ(*d_1*) ≠ σ(*d_2*). System* $\mathbb{B}$ *violates the Agreement Axis, yet it fully satisfies the Coherence Axis for all participating nodes, as the operational mutations of both local agents are preserved.*

By Lemma 1 and Lemma 2, Proposition 1 holds: satisfaction of one axis implies nothing about the satisfaction of the other. The Agreement Axis and the Coherence Axis are formally independent, orthogonal dimensions of distributed state evaluation.

Corollary 2 (Eventual Consistency Incomparability)
Eventual coherence and eventual consistency are incomparable: eventual consistency does not imply eventual coherence, and eventual coherence does not imply eventual consistency.

Proof Sketch. This orthogonality is guaranteed by Invariant 6 (Data Plane Non-Origination). If an update broadcast arrives at a non-originating device missing a local topological record, content materialization is structurally barred; the event is routed exclusively as an application signal. By applying Lemmas 1 and 2, clock advancement and tuple dominance decouple plane evolution, ensuring non-interference. The complete proofs for Lemmas 1, 2, and Proposition 4.5 are detailed in Appendix II.6, II.7, and II.8.

4.6 Liveness

Theorem 5 (Liveness)
If all messages are eventually delivered and all devices eventually cross synchronization boundaries, the system reaches a stable state in finite time.

The protocol must guarantee progress and prevent deadlock or indefinite blocking states during boundary crossings.

Proof Sketch. Liveness is guaranteed because the reconciliation loop evaluates state vectors sequentially over a finite set of keys without requiring distributed multi-phase locking or synchronous consensus rounds. The complete proof is detailed in Appendix II.6.

5. Conclusion

Eventual coherence is a synchronization model for replicated state in multi-device human-facing systems. The model rests on two foundational observations. First, resource existence and resource modification are categorically different operations with different causal properties, different timing requirements, and different correct reconciliation behaviors. Handling them uniformly, as most synchronization protocols do, produces systems that are simultaneously harder to build and less faithful to user intent. Second, for systems where devices represent independent agents operating in distinct causal contexts, convergence to identical state is not the correct criterion. Per-device causal validity is.

The formal contribution of this paper is the definition of eventual coherence as a correctness criterion that operates outside the classical distributed consistency hierarchy. Rather than measuring correctness by agreement, we decouple evaluation into two orthogonal dimensions: inter-replica agreement (the Agreement Axis) and local historical validity (the Coherence Axis) and prove that neither implies the other. Corollary 2 establishes the direct consequence: eventual coherence and eventual consistency are incomparable, with neither property subsuming the other. We have proved that the protocol guarantees per-device causal validity (thm. 1), convergence to causally valid states (thm. 2), update preservation (thm. 3), local deletion stability (thm. 4), and liveness (thm. 5). We have not claimed classical causal consistency, which requires global ordering properties incompatible with the intentional divergence eventual coherence permits, but something more appropriate for the systems we target: each device converges to a state that is correct given its own operational history, with causal ordering established through the happens-before relation rather than physical clock synchronization.

The practical contribution is a protocol built on two principles. Local deletion finality makes a device's deletion of a resource a permanent terminal state impervious to remote data plane operations. Server-level upsert semantics ensures that devices with active causal participation propagate their updates to third-party devices even in the presence of concurrent deletions. Together these principles resolve the delete-update conflict without coordination, without unbounded tombstone accumulation, without operational transformation, and without physical clock synchronization assumptions. The protocol is implementable over standard HTTP.

Eventual coherence is not the right model for every distributed system. Systems where replicas must agree on a single authoritative state, such as financial ledgers or inventory management, correctly prioritize consistency. The contribution is scoped to systems where replicas represent independent agents operating within distinct causal contexts,

and where preserving the validity of each agent's history is more meaningful than enforcing a single global state.

Future work falls into five areas. The relationship between synchronization boundary frequency and control plane staleness warrants formal characterization: how often must boundaries be crossed to bound divergence? The device lifecycle problem (decommissioning and deletion-record gc) requires treatment beyond what the proof-of-concept addresses. Extending eventual coherence to multi-server topologies is an open problem, requiring careful treatment of upsert semantics and Lamport synchronization across servers. The concurrent authorship problem, data plane semantics for multiple independent authors on one resource, requires a separate model; this work establishes the control-plane foundation such a model would rest on. Finally, and most consequentially, eventual coherence is a natural substrate for agent-to-agent coordination: autonomous agents face the same delete-update conflict formalized here, and the Orthogonality Proposition is arguably a stronger claim in that setting, since agents lack the informal coordination mechanisms humans use to resolve conflicts. As autonomous systems increasingly manage shared state without a human in the loop, a criterion that does not presuppose convergence stops being a theoretical nicety and becomes an operational requirement. Formalizing this substrate for multi-agent state management is the most direct extension of this work.

The deeper point concerns a foundational assumption in distributed correctness: should replicas be evaluated by their agreement with one another, or by the validity of their state relative to their own causal history? Classical models answer the former; eventual coherence introduces the latter not as a weakening of correctness, but a clarification of what it means. For an increasingly important class of systems, human or artificial agents acting asynchronously across distinct causal contexts, the second question is the right one. Correctness is not the elimination of divergence but the preservation of causally valid state. The question is not whether every replica shares the same history, but whether each has a history it is entitled to have.

APPENDIX I: Protocol Algorithmic Specifications

This appendix provides the formal algorithmic specifications, state transition handlers, and reference models for the eventual coherence protocol primitives defined in Section 3.

I.1 Local Operation Execution Primitives

Immediate local application of control plane and data plane operations prior to network propagation.

```
CREATE(r):
    Li := Li + 1
    r.id ← generate_unique_id()
    r.lamport ← (Li, di)
    r.created_by ← di
    r.content ← empty
    local_storage.put(r)

DELETE(r):
    Li := Li + 1
    r.is_deleted ← true
    r.deleted_at_lamport ← (Li, di)
    r.deleted_by ← di
    local_storage.put(r)

UPDATE(r, delta):
    if r.is_deleted then
        discard silently   // Invariant 5:
        return
    Li := Li + 1
    r.content ← apply(r.content, delta)
    r.lamport ← (Li, di)
    r.modified_by ← di
    local_storage.put(r)
```

DELETE does not remove the resource from local storage. It marks the resource as deleted and records the Lamport timestamp of the deletion for use during subsequent control plane reconciliation.

I.2 Real-Time Data Plane Synchronization Handlers

The routines below govern the real-time propagation of content updates from originating device to the server and the subsequent non-originating broadcast processing loop.

```
// Client execution following local UPDATE
on UPDATE(r, delta):
  apply_locally(r, delta)
  send_to_server(UPDATE_MESSAGE, r, r.clock)

// Non-originating client broadcast
processing loop
on receive UPDATE_BROADCAST(r) at device di
(not the originating device):
  Li := max(Li, r.clock.lamport) + 1
  r_local ← local_storage.get(r.id)

  // Invariant 6: Data Plane Non-Origination
  if r_local is null then
    signal_boundary_available()
    return

  // Invariant 5: Local Deletion Finality
  if r_local.is_deleted then
    discard
    return

  if compareLamport(r.clock, r_local.clock)
> 0 then
    r_local.messages ← r.messages
    r_local.clock ← r.clock
    local_storage.put(r_local)
```

I.3 Server-Side Message Reception Context

The tracking relay processes real-time data plane mutations and control plane pushes using non-evaluative tracking and data-plane restoration filters.

```
on receive UPDATE_MESSAGE(r, clock) from
device di:
  Ls := max(Ls, clock.lamport) + 1
  r_server ← server_storage.get(r.id)
  if r_server is null then
    server_storage.put(r)
    broadcast_to_connected_devices(r)
    return
  if r_server.is_deleted then
 // restore resource so other devices
receive this update (upsert)
    r_server.is_deleted ← false
  if compareLamport(r.clock, r_server.clock)
> 0 then
   r_server.messages ← r.messages
   r_server.clock ← r.clock
   server_storage.put(r_server)
   broadcast_to_connected_devices(r_server)

on receive CREATE_MESSAGE(r, clock) from
device di:
  Ls := max(Ls, clock.lamport) + 1
  r_server ← server_storage.get(r.id)
  if r_server is not null then return
  server_storage.put(r)

on receive DELETE_MESSAGE(record) from
device di:
  Ls := max(Ls, record.clock.lamport) + 1
    r_server ← server_storage.get(record.id)
  if r_server is null then return
  r_server.clock ← record.clock
  r_server.deleted_by ← di
  r_server.is_deleted ← true
  server_storage.put(r_server)
  server_storage.track_deletion(record)
```

I.4 Core Control Plane Boundary Reconciliation Engine

```
on synchronization_boundary():

// Step 1: Advance Lamport clock from
server tracking metadata
  Ls_current ←
fetch_from_server(LAMPORT_COUNTER)
  Li := max(Li, Ls_current) + 1

// Step 2: Fetch current server data plane
and control plane states
  server_resources ←
fetch_from_server(ALL_RESOURCES)
  server_deletions ←
fetch_from_server(ALL_DELETION_RECORDS)

// Step 3: Reconcile local state map
against server state
  for each r_server in server_resources do
    r_local ←
local_storage.get(r_server.id)
        r_del_record =
server_deletions.find(id == r_server.id)
 // Case 1: Local Deletion Finality
 if r_local exists and r_local.is_deleted
 then
   record ← DeletionRecord(r_local.id,
r_local.clock)
   send_to_server(DELETE_MESSAGE, record)
   continue

 // Case 2: Causal Participation Detection
Clock Primitives
  if r_del_record is not null
  then
    participated :=
       r_local is not null
       and
       compareLamport(r_local.clock,
         r_del_record.clock) > 0
    if not participated
    then
      local_storage.delete(r_server.id)

local_storage.track_deletion(r_del_record)
```

This algorithm defines the sequential evaluation of Case 1, Case 2, and Case 3 constraints when a client proxy cross-cuts a synchronization boundary ledger.

Else: participation confirmed, r_{local} retained as-is. No further action; the resource remains active locally and will be re-pushed only on subsequent local `UPDATE`

```
  // Case 3: Standard Last-Write-Wins (LWW) →
Data Merge
  else
    if r_local is null or
compareLamport(r_server.clock, r_local.clock)
> 0
    then
      local_storage.put(r_server)

// Step 4: Push independent local-only
resources to server
  for each r_local in local_storage.get_all()
  do
    if r_local.id not in server_resources and
not r_local.is_deleted then
      send_to_server(CREATE_MESSAGE, r_local,
r_local.clock)

      last_boundary_clock ← CoherenceClock(Li,
di)
```

I.5 Garbage Collection (Section 3.6)

```
/* Deletion records can be collected once all
devices have crossed synchronization boundary
& no device has retained resource */
on server_garbage_collect():
  for each record in
server_storage.get_all_deletion_records() do
    if
all_devices_crossed_boundary_since(record.clo
ck)
      then if
        no_device_retained(record.id)
        then
server_storage.delete_resource(record.id)
server_storage.purge_deletion_record(record.id
)
```

APPENDIX II: Formal Operational Verification and Extended Correctness Proofs

This appendix contains the complete, step-by-step mathematical derivations, case analyses, and inductive arguments for the theorems, lemmas, and propositions presented in Section 4.

II.1 Complete Proof of Theorem 1 (Per-Device Causal Validity)

We must show that for any operation o in H_i, the causal dependencies of o are respected in d_i's local state.

Case 1: o is a data plane operation (UPDATE). Data plane operations execute locally and immediately. By Invariant 1, operations within a synchronization epoch are totally ordered by Lamport timestamps. If o causally depends on a prior operation o' in H_i, then L(o') < L(o) by the Lamport happens-before relation [1]. Since d_i applies operations in Lamport timestamp order, o' is applied before o. The causal dependency is respected.

Case 2: o is a control plane CREATE operation. CREATE establishes a resource that subsequent operations may depend on. Since CREATE executes locally and increments Li before recording r in local storage, any operation o' in H_i that depends on r satisfies L(o') > L(CREATE(r)). The causal dependency is respected within H_i.

Case 3: o is a control plane DELETE operation. DELETE marks r as deleted locally. By Invariant 5 (Local Deletion Finality), no subsequent operation in H_i may target r. Therefore no operation in H_i causally depends on r existing after L(DELETE(r)). D_i's local state correctly reflects that r does not exist after the deletion.

Case 4: d_i crosses a synchronization boundary and discovers a deletion record for resource r at Lamport timestamp L_d.

Sub-case 4a: r_{local} is null, or $\tau_{local} \not\succ \tau_{delete}$ (r_{local}'s lexicographic timestamp does not strictly dominate the deletion record) and d_i has no operation in H_i whose Lamport tuple strictly post-dates the deletion. By Definition 7, d_i did not causally participate in r after the deletion. Accepting the deletion introduces no causal violation: d_i has no operations in H_i that causally depend on r existing after the deletion record.

Sub-case 4b: $\tau_{local} \succ \tau_{delete}$. d_i has at least one operation o in H_i with o.r = r whose lexicographic Lamport tuple strictly dominates the deletion record. By Definition 7, d_i causally participated in r after the deletion.

By Invariant 4 (Causal Participation Preservation), d_i retains r and o's operations on r after the deletion form a causally consistent sequence under the Lamport happens-before relation: each operation depends on the state of r as d_i observed it within its synchronization epoch.

Since d_i did not observe the deletion before performing these operations (if d_i had, Invariant 5 would have prevented further updates), these operations are causally valid. Note that the retained operations may be concurrent with the deletion in the happens-before sense: neither d_i's updates nor the deletion causally preceded the other. The Lamport ordering establishes a consistent total order between concurrent events without implying causal precedence [1]. This is the correct treatment of concurrency under the Lamport model.

In all cases, d_i's local state respects the causal dependencies in H_i. QED.

II.2 Complete Proof of Theorem 2 (Convergence to Causally Valid States)

We show that the reconciliation algorithm terminates in a stable state satisfying Definition 10. Let $T_{quiesce}$ be the time when all operations cease. Let T_{sync} be the time when all pending synchronization boundaries have been crossed. We claim that at time T >T_{sync}, every device's state is causally valid.

Step 1: Reconciliation terminates. Each reconciliation processes O(|R|) resources with constant-time comparisons. Since |R| is finite and no new operations occur after $T_{quiesce}$, reconciliation completes in finite time. Let T_{recon} be the time at which all devices complete reconciliation. T_{recon} is finite.

Step 2: The resulting state is causally valid for each device. Consider device d_i at time $T > T_{recon}$. For each resource r, exactly one of the following holds.

(a) d_i has locally deleted r. By Theorem 4 (Local Deletion Stability, proved below), d_i's deletion is stable across all synchronization boundaries. d_i's state is causally valid by Case 3 of Theorem 1.

(b) R carries no deletion record and d_i holds the latest version by Lamport-ordered last-write-wins. d_i's state is causally valid by Cases 1 and 2 of Theorem 1. Case (b) additionally covers devices that received but did not apply real-time broadcasts for resources outside their topological knowledge under Invariant 6; receipt triggered a boundary availability signal rather than content materialization. Such devices acquire the resource through the same Case 3 mechanism at their next synchronization boundary, with no loss of content, which remains durably held at S.

(c) R carries a deletion record and d_i's local Lamport tuple for r strictly dominates the deletion record lexicographically. d_i causally participated after the deletion and retains r. Thus, d_i's state is causally valid by Case 4b of Theorem 1.

(d) R carries a deletion record and d_i has no local record of r, or d_i's Lamport tuple for r does not strictly dominate the deletion record. d_i accepts the deletion and thus d_i's state is causally valid by Case 4a of Theorem 1.

In all cases, d_i's final state respects all causal dependencies in H_i, satisfying Definition 10. Cases (a) and (c) may produce permanently divergent states between devices. This divergence is the correct outcome under Definition 10, not a failure. Eventual coherence does not require cross-device convergence to identical state. It requires only that each device converge to a state that is causally valid for that device. QED.

II.3 Complete Proof of Theorem 3 (Update Preservation)

At the synchronization boundary, d_1 reconciles its local history against the server state. We evaluate by operation class:

Case 1: o is a CREATE(r) operation. The new resource topology is pushed to the server during reconciliation Step 4. Because resource identifiers are globally unique, r is cleanly established or verified at the server, preserving o's structural intent.

Case 2: o is an UPDATE(r, δ) operation. If r carries no server-side deletion record, o is preserved trivially via Lamport-ordered last-write-wins propagation. If r carries a deletion record τ_{delete}, the client applies Case 2 reconciliation (§3.4): if $\tau_o \succ \tau_{delete}$, d_i establishes active causal participation, retaining r and preserving o's modification; if $\tau_o \not\succ \tau_{delete}$, o fails to strictly dominate the deletion horizon and is safely superseded.

Case 3: o is a DELETE(r) operation. Under Case 1 of the reconciliation algorithm, d_i unconditionally re-asserts its local deletion to the server. By Theorem 4 (Local Deletion Stability), d_i maintains r as deleted, satisfying and preserving its destructive intent permanently. QED.

II.4 Complete Proof of Theorem 4 (Local Deletion Stability)

At each synchronization boundary crossing, the reconciliation algorithm evaluates Case 1 first and unconditionally: if r_{local} exists and r_{local}.is_deleted is true, the algorithm sends DELETE_MESSAGE to S and proceeds to the next resource without evaluating Cases 2 or 3. No path through the reconciliation algorithm sets r_{local}.is_deleted to false for a locally deleted resource. No incoming data plane update can modify a locally deleted resource (Invariant 5). Therefore once d_i has marked r as deleted, d_i's local state has r as deleted at every subsequent synchronization boundary crossing. QED.

II.5 Complete Proof of the Orthogonality Proposition (Section 4.5)

We prove Lemma 1, Lemma 2, and the main orthogonality result in sequence.

Proof of Lemma 1 (Agreement Without Coherence). System A must satisfy the Agreement Axis by producing a unique, unified terminal state $\sigma(d_1) = \sigma(d_2)$. Under concurrent operations DELETE(r) on d_1 and UPDATE(r, δ) on d_2, any deterministic reconciliation rule in A must select one of exactly two terminal states: the deleted state or the active updated state. If A selects the deleted state, the committed update δ is erased from d_2's materialized state; d_2 has a committed operation in its local history whose structural intent is not preserved. If A selects the active updated state, d_1's committed deletion is reversed; d_1 has a committed operation in its local history whose structural intent is not preserved. In either execution path, system A violates the Coherence Axis for at least one device. Lemma 1 is established.

Proof of Lemma 2 (Coherence Without Agreement). System B operates under the eventual coherence protocol. By setup, d_2's local Lamport counter has been advanced past τ_delete = $\langle l_d, d_d \rangle$ prior to issuing UPDATE(r, δ), so τ_local $\succ \tau_{delete}$. Upon crossing a synchronization boundary: d_i's reconciliation encounters Case 1 (Local Deletion Finality) unconditionally. d_i sends DELETE_MESSAGE to S and maintains $\sigma(d_i) = \varnothing$. d_2's reconciliation encounters the deletion record for r and evaluates the participation predicate. Because τ_local $\succ \tau_{delete}$, d_2 establishes active causal participation, retains r, and propagates its update, materializing $\sigma(d_2) = \{r \mapsto \delta\}$.

As $t \to \infty$, no subsequent synchronization boundary crossing modifies either state: d_i re-asserts the deletion unconditionally under Case 1, and d_2 re-asserts its update under Case 2. The states remain permanently diverged: $\sigma(d_i) \neq \sigma(d_2)$. System B violates the Agreement Axis. Yet the Coherence Axis is fully satisfied for all participating nodes: d_1's deletion is preserved and d_2's update is preserved, with each device's state causally valid given its local history. Lemma 2 is established.

Proof of Proposition 1 (Orthogonality). The Agreement Axis (Definition 8) is the formal expression of eventual consistency within this model. By Lemma 1, a system satisfying the Agreement Axis (and therefore eventual consistency) may fully violate the Coherence Axis; eventual consistency does not imply eventual coherence. By Lemma 2, a system satisfying the Coherence Axis on all devices may maintain permanent state divergence, violating the Agreement Axis and therefore eventual consistency; eventual coherence does not imply eventual consistency. Since neither property implies the other, the two are incomparable. QED.

II.6 Complete Proof of Theorem 5 (Liveness)

Assume eventual message delivery and eventual synchronization boundary crossings. We show that the system reaches a fixed point. Let $T_{quiesce}$ be the time when

all operations cease. After $T_{quiesce}$, no new operations enter any device's local history. Each subsequent synchronization boundary crossing makes deterministic decisions based on Lamport timestamps and local history. Since no new operations occur after $T_{quiesce}$, these inputs do not change. By Invariant 2 (Control Plane Idempotence), repeated synchronization boundary crossings after $T_{quiesce}$ produce identical decisions.

By Theorem 4 (Local Deletion Stability), deletion decisions made at synchronization boundaries are stable. By Invariant 3 (Data Plane Convergence), data plane state converges to the version with the highest Lamport timestamp among non-deleted replicas. Therefore there exists a time $T > T_{quiesce}$ after which no device's state changes at any synchronization boundary crossing. The system has reached a fixed point.

This fixed point may include permanently divergent states between devices. A device that has deleted a resource and a device that has retained it will remain divergent indefinitely. This is a stable and correct terminal state, not a failure mode. The liveness property guarantees termination of the reconciliation process, not convergence to identical state. QED.

APPENDIX III: Related Work

Eventual coherence sits at the intersection of several research traditions. We survey each in turn, positioning our contribution precisely against the closest related work.

III.1 Limits of Convergent Approaches

The Lineage of Optimistic Replication and the Limits of Convergent Consensus

The foundations of modern collaborative software rest upon early optimistic replication frameworks designed to maximize availability in weakly connected environments. Chief among these is Bayou [1], which pioneered the use of per-replica operation logs, tentative executions, and background anti-entropy protocols to resolve concurrent writes without synchronous coordination. To ensure that temporary divergence does not degrade into permanent state fracturing, Bayou relies on a designated primary site to assign a definitive, monotonically increasing commit sequence number to every write operation. Replicas continuously roll back tentative local states and replay operations in accordance with this global sequence, ensuring that all nodes eventually arrive at an identical ledger.

Eventual coherence differs fundamentally from these early optimistic replication models in its definition of distributed correctness. Bayou assumes that all replicas must ultimately converge to an identical state determined by a primary commit site, treating concurrent divergence as an expected transient state during anti-entropy and reconciliation to be eliminated through global serialization. In contrast, eventual coherence treats concurrent deletion and modification not as a conflict requiring resolution, but as potentially intentional bifurcation. Where Bayou treats permanent divergence as a correctness failure, eventual coherence preserves the validity of both deletion and continued participation within their respective causal histories, elevating certain forms of permanent divergence to the correct outcome for human-facing systems.

This conceptual divergence manifests in the underlying mechanics of the reconciliation plane. Because Bayou's correctness criterion is identity, it enforces a total order that inevitably creates an operational winner and loser when confronting concurrent modifications to the same entity; a user's local, tentative deletion can be summarily overturned and resurrected by a subsequent commit sequence from the primary site. Eventual coherence replaces this centralized serialization with asymmetric reconciliation rules executed at synchronization boundaries via Lamport timestamps. By partitioning operations into discrete planes and enforcing Local Deletion Finality, our protocol guarantees that a device's choice to terminate its participation with a resource is structurally irreversible, allowing divergent data-plane timelines to coexist without violating causal validity.

The asymmetric control plane propagation model this enables, immediate push but boundary-only pull, has been observed in mobile computing systems [2, 54] and client-server synchronization protocols [8, 53], but has not previously been formalized as a correctness model with precise reconciliation semantics and provable guarantees. The observation that devices push control plane events immediately while deferring their pull to synchronization boundaries is, to our knowledge, novel as a formal protocol element with defined correctness guarantees.

III.2 Eventual Consistency and Large-Scale Systems

The practical systems literature on eventual consistency is anchored by Dynamo [10], which demonstrated that last-write-wins timestamp ordering and vector clock-based reconciliation could support high availability at scale while tolerating network partitions. Cassandra [11] and Riak [55] extended this work, with Riak subsequently adding CRDT support to address the semantic limitations of pure LWW reconciliation. These systems treat all operations as equivalent writes resolved through timestamp ordering or causal dependency tracking. This uniform treatment is appropriate when the application domain does not distinguish between operations that

establish resource existence and operations that modify resource content. Eventual coherence notes that for an important class of systems this distinction is fundamental, and that treating topology and content operations uniformly produces reconciliation semantics that are simultaneously harder to implement correctly and less faithful to user intent.

Dynamo's use of physical timestamps for last-write-wins is vulnerable to clock skew [1]. Eventual coherence uses Lamport timestamps for all ordering decisions, properly grounding the protocol in the happens-before relation and eliminating dependence on physical clock synchronization. For data plane operations, eventual coherence also uses last-write-wins, but under Lamport ordering rather than physical time, inheriting Dynamo's simplicity while correcting its formal grounding.

III.3 Conflict-Free Replicated Data Types and Operational Transformation

Conflict-Free Replicated Data Types (CRDTs) [14, 15, 16] offer a principled framework for coordination-free eventual consistency by leveraging mathematical properties that guarantee convergence regardless of network routing, delay, or operation reordering. State-based CRDTs [14, 20] evaluate state mutations as monotonic joins over a bounded join-semilattice, while operation-based CRDTs [15, 49] achieve the same end state by delivering commutative operations over a causally ordered transport [50]. The structural limitation of CRDTs when applied to the multi-device delete-update problem is not merely an engineering concern regarding state growth; it is a fundamental semantic mismatch dictated by their underlying algebraic invariants. By definition, a state-based CRDT requires that for any two concurrent states x and y, the merge operation must compute the least upper bound (the join) within a semilattice: $x \sqcup y$. This algebraic formulation enforces a critical system invariant: if a network achieves quiescence, replicas must converge to an identical, uniform state ($x \sqcup y = y \sqcup x$). Consequently, a CRDT lattice is structurally incapable of representing a permanent causal split.

When confronted with a concurrent delete-update pair, a CRDT must enforce a deterministic, global policy to maintain lattice validity, either by prioritizing deletion globally (as in standard Observed-Remove Sets [15]) or by preserving the updated value globally within a multi-value container (an MV-CRDT). Neither approach can satisfy the requirements of human-facing multi-device synchronization. If the system defaults to a "delete wins" policy, active updates performed in good faith by a user on a separate device are permanently erased. Conversely, if an MV-CRDT is employed to preserve both the deleted state and the concurrent update, the protocol merely defers the conflict to the application layer, forcing every device on the network to track, store, and eventually resolve the multi-value set. This introduces a subtle but severe violation of Local Deletion Finality (Invariant 5): a device that has explicitly deleted a resource is structurally barred from forgetting it. The deleting device is forced to maintain a persistent causal relationship with the dead resource, processing remote data-plane updates it has already opted out of, simply to prevent the global lattice from fracturing.

Furthermore, while modern CRDT literature proposes various optimizations for deletion record garbage collection and delta-state minimization [21, 28], these mitigation strategies invariably break the pure, zero-coordination deployment model. They rely on environmental assumptions such as physical clock synchronization, bounded network delivery latencies, or periodic stable-vector consensus protocols to safely purge stale metadata without risking state regression. Under the eventual coherence model, we reject both the coordination tax of garbage collection and the semantic tyranny of the join-semilattice. By decoupling the control and data planes, eventual coherence permits a radical departure from CRDT invariants: it allows the deleting device to enter a stable, terminal state where it completely purges and ignores the resource, while allowing the retaining device to preserve its data plane mutations. The system achieves local causal validity without forcing a uniform least upper bound across divergent human intents.

Operational transformation (OT) [45, 46] similarly preserves concurrent updates but requires the maintenance of complex operation logs and the evaluation of transform functions whose mathematical correctness is notoriously fragile [47]. Systems utilizing OT at scale, such as Jupiter [52], introduce significant architectural and computational complexity to resolve concurrency. Eventual coherence bypasses the algorithmic overhead of OT entirely. Rather than attempting to transform or synthesize conflicting concurrent operations, our protocol partitions operations into planes with asymmetric reconciliation rules, reducing the complex delete-update paradox to a localized, deterministic Lamport timestamp comparison executed at synchronization boundaries.

III.4 Control Plane and Data Plane Separation

While the separation of control and data planes is an established primitive in networking and distributed storage, existing architectures leverage this isolation to optimize or enforce uniform convergence. We are unaware of a prior synchronization model that utilizes plane separation to formalize intentional, permanent divergence as a valid correctness criterion for human-facing multi-device state. Software-defined networking [44] separates the control plane, which establishes network topology and routing policy, from the data plane, which forwards packets according to that policy. These planes operate at different timescales with different consistency requirements: control plane updates are relatively infrequent and may tolerate higher latency, while data plane operations must be fast and local. This is structurally identical to the distinction eventual

coherence formalizes between CREATE and DELETE operations on the one hand and UPDATE operations on the other.

Burns et al. [56] describe a related distinction between desired state, declared through control plane operations and reconciled by controllers, and runtime state, which reflects the current operational behavior of the system. Version control systems such as Git maintain a similar separation between refs, which establish the topology of the commit graph, and file contents, which are modified through commits. Eventual coherence formalizes this separation as a synchronization model for replicated state. The contribution is not the separation itself, which has clear antecedents in these systems traditions, but its formalization as a correctness criterion with precise reconciliation semantics, Lamport clock grounding, and provable invariants including Local Deletion Finality and Causal Participation Preservation.

III.5 The CAP Theorem and Availability

Brewer's CAP theorem [7], formalized by Gilbert and Lynch [43], establishes that a distributed system cannot simultaneously provide consistency, availability, and partition tolerance. Systems that require coordination for conflict resolution, including two-phase commit [4] and Paxos-based consensus [5, 6, 31], sacrifice availability under partition in exchange for consistency guarantees. Eventual coherence accepts this tradeoff explicitly, choosing availability. The contribution goes beyond simply accepting the tradeoff: the paper argues that for the class of systems it targets, consistency is not the correct objective even when achievable. The CAP theorem frames the choice as a reluctant concession to the realities of distributed systems. Eventual coherence reframes it as a principled design decision grounded in the semantics of the operations and the nature of the agents performing them. Forcing convergence between a device that has deliberately deleted a resource and a device that has retained it through active causal participation does not improve correctness. It imposes an arbitrary resolution on a situation that never required one.

Appendix IV. Proof of Concept and Implementation

An experimental proof-of-concept system that demonstrates the practical realizability of the eventual coherence protocol using standard web technologies is implemented in the Polyglot codebase, a distributed model workspace used for memory modeling. This implementation uses a multi-device AI conversation management system which allows a user to maintain chat histories across multiple devices and AI providers. It is not a production system and is not presented as one. Its purpose is to demonstrate that the protocol can be realized without specialized distributed systems infrastructure, and to surface engineering considerations that a full reference implementation will need to address. A full reference implementation is in progress.

IV.1 System Architecture

Polyglot consists of three components. Client devices run a web app that stores state in IndexedDB, a browser-native key-value store, utilizing independent object stores for resources, synchronization metadata, and historical deletion records. This provides persistent storage with ACID transactions within a single device, making it suitable for maintaining local replicas. Database initialization is deferred until first access via a lazy initialization guard, avoiding main-thread blocking during component mounting. Each browser context constitutes a device in the sense of Definition 4, maintaining independent local history and its own Lamport counter, and crossing synchronization boundaries independently. The server is a Bun.js HTTP service that maintains persistent current state for all resources and broadcasts data plane updates to connected devices in near-real-time. As established in §2.5, S is a shared communication and persistence layer; it does not maintain per-device state and has no knowledge of which devices have performed which local operations.

Protocol implementation spans both client and server, realizing the algorithms from Section 3. Client side is organized around a set of dedicated modules: db.ts, the storage adapter which owns three IndexedDB object stores for data plane state (chats), control plane causal horizons (deletions), and device identity/Lamport counter persistence (metadata); conversationSync.ts (write coordination, stamping each mutation with a fresh Lamport tick before persistence); ReconciliationEngine.ts (the synchronization boundary algorithm, evaluating control plane and data plane reconciliation across discrete phases); CoherenceClock.ts (Lamport clock state), and ConversationStateManager.ts (the UI-facing orchestrator, which makes no protocol decisions itself and delegates all writes and reconciliation to the modules above). A storage compatibility shim remains for callers not yet migrated to the current module boundaries.

IV.2 Data Structures

Each conversation resource r can be represented as a LoopResource, holding content and a structured Lamport clock tuple:

```
interface LoopResource {
  id: string;
  title: string;
  messages: Message[];
     lastMutationLamport:  ClockTuple;     //
{ lamport: number, deviceId: string }
  createdAt: Date;
  lastModified: Date;
}
```

Deletion state is not a field on LoopResource. It is represented by a separate DeletionRecord, held in a distinct IndexedDB object store (deletions), structurally isolated from the chats store holding live resources:

```
interface DeletionRecord {
  id: string;
  deletedAtLamport: ClockTuple;
}
```

This separation is a direct structural realization of the plane distinction argued for in Section 1: a resource's presence in chats versus deletions is itself the control-plane fact, rather than a flag inspected on a unified record. A resource id present in neither store is unambiguously unknown to the device; a resource id present only in deletions is unambiguously deleted; the two cases cannot be confused, which is the property Invariant 5's retention requirement depends on.

Physical timestamp fields (createdAt, lastModified) are retained for human-readable display only. They are not used in any protocol decision. All ordering decisions use the Lamport tuple exclusively. Deletion records are held in a separate IndexedDB object store (deletions) from live resources (chats), so Invariant 7's persistence requirement is a consequence of this structural separation rather than a field-preservation discipline: an upsert restoring a resource writes to the chats store and has no code path that can reach, let alone modify, a DeletionRecord in the deletions store. The deletion horizon therefore cannot be cleared as a side effect of a data plane write, by construction rather than by convention. This separation is what enables correct participation decisions at synchronization boundaries even after a resource has been restored by an upsert. The Lamport counter for each device is stored persistently in a dedicated metadata object store, separate from both resource and deletion state. This ensures the counter survives application restarts and page refreshes. A device that restarts without its Lamport counter would be unable to correctly establish happens-before ordering with prior operations.

IV.3 Local Operation Execution

Write operations are coordinated through conversationSync.ts, which stamps each mutation with a fresh Lamport tick before delegating persistence to db.ts:

```
export async function
saveConversation(resource: LoopResource):
Promise<ConversationSyncResult> {
  const tau =
CoherenceClock.getInstance().tick();
  const stampedResource: LoopResource = {
    ...resource,
    lastMutationLamport: tau,
    lastModified: new Date()
  };
  const wasSaved = await
polyglotDb.saveResource(stampedResource);
  return { success: true, changed: wasSaved,
error: null };
}
```

```
export async function
deleteConversation(conversationId: string):
Promise<ConversationSyncResult> {
  const tau =
CoherenceClock.getInstance().tick();
  const record: DeletionRecord = { id:
conversationId, deletedAtLamport: tau };
  await
polyglotDb.deleteResource(conversationId,
record);
  return { success: true, changed: true,
error: null };
}
```

Invariant 5 enforcement is not implemented in this coordination layer. It lives in db.ts's saveResource, which checks for an existing local deletion record before writing and silently declines the write (returning false) if one is present; the caller observes this via the changed field of the returned result rather than an exception. deleteResource performs the resource removal and the deletion record write as a single atomic IndexedDB transaction spanning both the chats and deletions stores, since the two writes must not be separable: a resource removed without a corresponding record would be indistinguishable, on next read, from a resource that was never created.

IV.4 Data Plane Synchronization

The client's broadcast receive handler lives in ConversationStateManager, implementing the Invariant 5/6 distinction directly against db.ts rather than through the write-coordination layer, since this check has no protocol decision to enforce, only a fact to observe:

```
private async handleBroadcast(broadcast:
LoopResource): Promise<void> {
CoherenceClock.getInstance().observe(broadcas
t.lastMutationLamport);

  const localRes = await
polyglotDb.getResource(broadcast.id);
  const localDel = await
polyglotDb.getDeletionRecord(broadcast.id);

  if (!localRes && !localDel) {
    this.signalBoundaryAvailable();
    return; }
  if (localDel) {
    return; }

  if (localRes &&
broadcast.lastMutationLamport.lamport >
localRes.lastMutationLamport.lamport) {
    await polyglotDb.saveResource(broadcast);
    await this.loadConversations();
  }
}
```

The three-way branch directly reflects the store separation of IV.2: an id absent from both stores is unknown (Invariant 6, signal boundary availability); an id present in deletions is locally deleted (Invariant 5, discard unconditionally); an id present only in chats is active and is updated in place if the broadcast's Lamport tuple dominates the local one.

IV.5 Control Plane Reconciliation

Synchronization boundary reconciliation is implemented in ReconciliationEngine.reconcileBoundary, evaluated as two sequential phases operating over the incoming server deletion and resource sets:

```
// Phase 1: Control plane — process incoming
deletion records first,
// establishing causal horizons before data
plane resources are evaluated.
for (const remoteDel of incomingDeletions) {
  const localRes = await
this.db.getResource(remoteDel.id);
  const localDel = await
this.db.getDeletionRecord(remoteDel.id);

  if (localDel) {
    if
(compareLamport(localDel.deletedAtLamport,
remoteDel.deletedAtLamport) > 0) {
      await
this.db.saveDeletionRecord(remoteDel);
    }    continue;  }

  if (localRes) {
    if
(compareLamport(localRes.lastMutationLamport,
remoteDel.deletedAtLamport) > 0) {
      continue; // local mutation post-dates
deletion: causal participation, deletion
discarded
    }
    await
this.db.saveDeletionRecord(remoteDel);
    await
this.db.deleteResource(remoteDel.id,
remoteDel);
  } else {
    await
this.db.saveDeletionRecord(remoteDel);
// Invariant 5: unknown resource record kept
  }
}
// Phase 2: Data plane — process incoming
resource mutations.
for (const remoteRes of incomingResources) {
  const localDel = await
this.db.getDeletionRecord(remoteRes.id);
  if (localDel) continue; // Invariant 5:
per-device finality, unconditional

  const localRes = await
this.db.getResource(remoteRes.id);
  if (!localRes ||
strictlyDominates(remoteRes.lastMutationLampo
rt, localRes.lastMutationLamport)) {
    await this.db.saveResource(remoteRes);
```

The reconciliation engine also implements a third phase governing deletion-record garbage collection against the server's active resource id set, described in Section 3.6.

IV.6 Engineering Considerations

Lamport counter persistence. The Lamport counter must survive application restarts and page refreshes. Storing it in a dedicated IndexedDB store separate from conversation state ensures it is not accidentally cleared when conversations are modified. The counter must also be advanced correctly on server state fetch: receiving the server's current Lamport value and taking the maximum before incrementing ensures the device's counter reflects the global state of the system.

Lamport counter and device identity. Each device requires a stable, unique identifier to serve as tie-breaker in Lamport ordering. In the proof of concept this is generated on first launch and stored in localStorage. A production implementation will use a more robust mechanism.

Synchronization boundary. The proof of concept treats application initialization as a synchronization boundary. This is one valid choice among many. Other implementations may define additional synchronization boundaries at explicit user actions, periodic intervals, or application-defined state transitions. The protocol is agnostic to this choice provided synchronization boundaries occur with sufficient frequency to bound the staleness of control plane state.

Dirty state tracking. When a data plane synchronization fails, conversations are marked dirty via a separate IndexedDB. On the next synchronization boundary, they're pushed to S before reconciliation fetch, ensuring local data plane state is reflected in S before reconciliation decisions are made.

Concurrent local contexts. Multiple browser tabs on the same device share a single IndexedDB instance, including the Lamport counter store. IndexedDB transactions serialize access to the counter, preventing duplicate Lamport values for a single device. The reconciliation algorithm assumes a single active context per device at each synchronization boundary. Production implementations should define explicit device boundary semantics for multi-tab scenarios.

Deletion record preservation. The most operationally critical implementation detail, formalized as Invariant 7, is that the server must never clear deletedAtLamport when applying a data plane upsert. This field is the mechanism by which reconciling devices learn that a deletion occurred and determine their causal participation. Its accidental omission would cause devices to treat upserted resources as never-deleted, preventing correct participation decisions.

Local deletion record retention. deleteConversation calls db.conversations.put(conv) rather than db.conversations.delete(conv.id). This is a protocol requirement, not an implementation convenience: the deletion record must remain in IndexedDB to allow the conversationUpdated listener to distinguish a deleted resource from an unknown one. The two cases produce different responses (silent discard versus boundary availability signal) but collapse to the same missing-record result if the deletion record is removed. A local deletion record may be purged only when a synchronization boundary reveals the resource absent from S's resource list, indicating that server-side garbage collection is complete and no future broadcasts for that resource id will arrive.

IV.7 Code Availability

The Polyglot proof-of-concept implementation conforming to the formal model of Section 2 is available under the MIT license. The repository it has been implemented on (github.com/forestmars/polyglot) is a model workspace intended for LLM memory experimentation and is under active development.

APPENDIX V: Evaluation

We evaluate the eventual coherence protocol along two dimensions: correctness under the conflict scenarios its formal model addresses, and a preliminary characterization of performance properties. We do not claim benchmark superiority over alternative systems. The proof-of-concept nature of the Polyglot implementation precludes rigorous comparative evaluation. What we demonstrate is that the protocol behaves correctly under its target scenarios and that its performance characteristics are consistent with the complexity analysis of Section 3.7.

V.1 Experimental Setup

Our experiments use three simulated devices, implemented as separate browser contexts sharing no local state, communicating with a single server instance. A fourth context is used for new device experiments. Each browser context maintains an independent IndexedDB instance, its own Lamport counter, and crosses synchronization boundaries independently. Synchronization boundaries are triggered manually to allow precise control over the sequencing of conflict scenarios. We report Lamport timestamps alongside wall clock times where relevant. Lamport timestamps are the load-bearing ordering mechanism in the protocol. Wall clock times are provided for readability only.

All experiments run on a single machine with an Intel Core i7 processor and 16GB RAM. Clients runs Chrome 149 & Brave 1.91; server runs Bun.js 1.3.2. Each experiment is repeated 10 times and we report median values.

V.2 Correctness Evaluation

We validate the protocol's correctness properties through systematic testing of scenarios addressed by the formal model. Each experiment targets one or more theorems or invariants from Section 4.

Experiment 1: Core scenario (delete, update, and third-party propagation.) Device A deletes conversation C at Lamport timestamp Ld=50. A pushes the deletion to the server immediately. Device B, operating within its current synchronization epoch and unaware of the deletion, adds a message to C. B's Lamport clock, advanced by prior activity on other resources, is at 80. B's update carries Lamport timestamp Lu=81. The server receives B's update for the deleted C, applies the upsert (restoring C, preserving deletedAtLamport=50), and broadcasts the restored C to Device C. Both A and B then cross synchronization boundaries.

Result. Device C receives B's update via the upsert broadcast. Device B crosses its synchronization boundary; τ_{local}=(81, B) $\succ$ τ_{delete}=(50, A), so B retains C through causal participation. Device A crosses its synchronization boundary; Case 1 triggers (A has C locally deleted), A re-asserts the deletion to the server, and A maintains C as deleted. This confirms Theorem 2 and validates the upsert propagation mechanism: third-party devices receive updates from active participants regardless of deletion state.

Experiment 2: Local Deletion Finality under upsert pressure. Device A deletes conversation C at Lamport timestamp Ld=50. Device B updates C repeatedly at Lamport timestamps 60, 70, and 80, triggering three successive upserts at the server and three restoration broadcasts to A.

Result. A discards all three restoration broadcasts under Invariant 5. A's local state has C as deleted throughout. At A's next synchronization boundary, Case 1 triggers, A re-asserts the deletion, and the server marks C as deleted again. This confirms Theorem 4: A's deletion is stable across all synchronization boundary crossings and cannot be overridden by remote data plane operations regardless of their volume or Lamport values.

Experiment 3: Default to deleted for new device. Device A deletes conversation C at Lamport timestamp Ld=50. Device B updates C, triggering a upsert at the server (C restored, deletedAtLamport=50 preserved). New Device D connects and crosses its first synchronization boundary. D has no local record of C and no causal participation.

Result. D's reconciliation algorithm reaches Case 2 for C: deletedAtLamport=50 is set and r_local is null, so the participation check fails (no strict lexicographic dominance possible with no local state). D defaults to deleted and does not acquire C. This confirms the default-to-deleted rule: a device with no causal participation accepts the deletion regardless of the server's current restoration state.

Experiment 4: Lamport participation threshold, including equal-counter tie-break. Three sub-experiments establish the boundary of the participation check, including the tie-break behavior introduced by the full lexicographic comparison.

- Sub-experiment 4a (participation confirmed). Device B has been active across multiple conversations, advancing B's Lamport clock to 120. Device A deletes conversation C at Lamport timestamp Ld=100. B updates C at Lamport timestamp Lu=121. At B's synchronization boundary: τ_local=(121, B) ≻ τ_delete=(100, A). B retains C.

- Sub-experiment 4b (non-participation confirmed). Device B has been inactive. B's Lamport clock is at 40. Device A deletes C at Lamport timestamp Ld=100. B updates C at Lamport timestamp Lu=41. At B's synchronization boundary: τ_{local}=(41, B) $\nsucc$ τ_{delete}=(100, A). B accepts the deletion.

- Sub-experiment 4c (equal-counter tie-break, defaults to deleted). Device B updates C with Lamport counter l=75. Device A deletes C with Lamport counter l=75. At B's

synchronization boundary: $l_{local} = l_{delete} = 75$. Since B < A lexicographically (B ≤ A), $\tau_{local} \not< \tau_{delete}$, and B defaults to deleted. This confirms deterministic resolution of perfectly concurrent operations in favor of the destructive control plane operation, without cross-device coordination.

Result. The full lexicographic Lamport participation check correctly distinguishes causal participation from non-participation and resolves equal-counter ties deterministically. This validates Definition 7 as implemented.

Experiment 5: Control plane boundary delay. Device A creates conversation X within its synchronization epoch. Device B creates conversation Y within its synchronization epoch. Neither has crossed a synchronization boundary. Both then cross synchronization boundaries.

Result. Both X and Y appear on both devices after synchronization boundary crossings. New resources created within a synchronization epoch propagate correctly at the boundary through Step 4 of the reconciliation algorithm. This confirms property (1) of the Problem Statement in Section 2.7: devices perform operations continuously within epochs without coordination. These five experiments validate the protocol's correctness properties: per-device causal validity (Theorem 1), convergence to causally valid states (Theorem 2), update preservation (Theorem 3), local deletion stability (Theorem 4), and the default-to-deleted rule.

V.3 Performance Characteristics

We report preliminary, informally observed performance characteristics consistent with the complexity analysis of Section 3.7. Measurements were collected using instrumented logging in the client reconciliation layer and Bun.js server event timestamps under local execution conditions during development. These figures are illustrative observations, not validated benchmarks, and are not asserted by the accompanying automated test suite.

Lamport counter overhead. Lamport counter increment and persistence (IndexedDB write) adds negligible overhead relative to network round-trip time (<1ms observed) and does not materially affect data plane latency.

Latency (informal observation). During development, data plane updates completed locally in under 5ms and propagated to the server in approximately 100 to 150ms over a local network connection, consistent with network round-trip time rather than protocol overhead. Control plane reconciliation at a synchronization boundary was observed to take approximately 50 to 200ms depending on the number of conversations, consistent with the O(|R|) complexity of Section 3.7. These figures were recorded manually under the local network conditions described in Section V.1 and are not asserted as validated benchmarks; the accompanying test suite verifies correctness and scale, and not these specific timing figures.

Storage. In a test dataset of 137 conversations with between 5 and 50 messages each, total storage per device was 2.3MB. The lamport and deletedAtLamport fields added under 2KB of overhead across the dataset. Deletion records added less than 1KB of additional metadata.

V.4 Comparison with Alternative Approaches

We implemented simplified versions of three alternative approaches within the same proof-of-concept harness to characterize behavioral differences. These are not rigorous comparative benchmarks. The alternative implementations are not optimized and the workload is not designed to stress them systematically.

Last-Write-Wins. We modeled a simplified Last-Write-Wins scheme in which DELETE is treated as a timestamped write operation competing with UPDATE on equal terms, using Lamport timestamps for ordering. The model is a lightweight simulation (four scenarios covering dominant updates, dominant deletes, tied-counter tie-breaks, and stale-operation handling), not a full alternative implementation of the protocol intended to characterize behavioral divergence from Invariant 5.

Result. In this model, deletion/update compete under the same ordering. If B's update carries a higher Lamport timestamp than A's deletion, B's update wins globally: C is restored on all devices including A. A, which explicitly deleted C, sees C restored. There is no equivalent of Invariant 5: the deleting device loses its deletion when outpaced by a more active retaining device. Such behavior is confusing and contrary to user expectation, since the deleting device loses its own deletion without any indication of why.

CRDT (Observed-Remove Set). We implemented deletion using an observed-remove set. Resources accumulate deletion tombstones that are never garbage collected without external coordination.

Result. After 1000 operations including 100 deletions, storage grew to 4.1MB compared to 2.3MB for eventual coherence, with tombstones comprising 45% of total storage.

The deletedAtLamport field in eventual coherence serves a similar bookkeeping function but is a single integer per deleted resource, eligible for garbage collection once all devices have reconciled. Eventual coherence achieves true deletion without unbounded storage growth.

Coordination-based (Two-Phase Commit). We implemented a two-phase commit protocol requiring acknowledgment from all participating devices before a deletion is finalized, with Lamport-ordered acknowledgments.

Result. When any device was slow to cross a synchronization boundary, deletions blocked indefinitely. In informal testing, average deletion latency was ~800ms, compared to <5ms for local deletion in eventual coherence; this figure is an informal observation under local test conditions, not a validated benchmark. Alternate system violated availability during control plane boundary delays, consistent with the CAP theorem analysis of Section 2.7.

V.5 Observational Notes

During informal use of the Polyglot proof of concept across multiple personal devices over approximately one month, no instances of data loss were observed. The synchronization boundary behavior matched the protocol's stated properties: resources modified within a synchronization epoch were retained after boundary crossings when causal participation was established, and resources not modified were correctly updated or deleted based on server state.

One interaction pattern confirmed Theorem 4 in practice. A conversation deleted on one device remained deleted on that device across multiple synchronization boundary crossings, even while another device continued adding messages to the same conversation. The retaining device continued to function normally. Third-party devices received updates from the retaining device through the upsert mechanism. Deleting device state was unaffected throughout.

The default-to-deleted behavior for new devices was also observed. A new browser context connecting for the first time did not acquire conversations carrying deletion records, regardless of whether those conversations had been restored by another device's data plane updates.

V.6 Limitations

Scale and Causal Horizons. The protocol's reliance on scalar Lamport timestamps rather than vector clocks introduces a distinct causal horizon boundary for isolated operations. If a device executes a local update while its local logical clock is lagged relative to a remote deletion event ($l_{local} \leq l_{delete}$), the reconciliation algorithm will mathematically categorize the update as superseded. This represents the formal limits of the protocol's correctness framework: historical validity is guaranteed only for devices operating within the active causal horizon of the cluster. A device that fails to advance its local Lamport counter relative to the cluster's control plane mutations effectively drops out of the causal sequence, and its concurrent mutations are treated as causally inert upon reconciliation.

Network. All experiments used local network conditions with round-trip times of ~100 to ~500ms. Wide-area network conditions with higher latency or more complex partition patterns may surface behaviors not observed in these experiments, though the protocol's correctness properties are not dependent on network latency bounds.

Workload generality. The workload is representative of single-author conversation management but may not generalize to domains with substantially higher write contention, more frequent control plane operations, or different ratios of data plane to control plane activity. The protocol's guarantees are stated over resources whose mutation history is causally linear, which should not be mistaken for a coverage gap; concurrent multi-author operations fall outside the domain by construction rather than by omission. In the LLM conversation domain this is not an arbitrary restriction, since each response is generated conditioned on the visible context at generation time. Concretely, Lamport-ordered last-write-wins at the resource level would discard operations, violating Property 6, and we believe the deeper issue is semantic rather than mechanical: a transcript formed by interleaving turns from distinct causal contexts may not correspond to any conversation either device actually had. Supporting concurrent multi-author history is a further application of this framework: causal-history tracking—enabling devices to distinguish valid from superseded mutations—is exactly what a branching model would need to distinguish a coherent merge from an incoherent one. We leave the branching extension itself to future work.

Comparative evaluation. Alternative implementations evaluated in Section 6.4 are not optimized and comparison is qualitative rather than quantitative. Rigorous comparative benchmarking against production-quality implementations of alternative approaches is left to future work.

References:

[1] Lamport, L. 1978. Time, clocks, and the ordering of events in a distributed system. Commun. ACM 21, 7 (July 1978), 558–565.

[2] Terry, D. B., Theimer, M. M., Petersen, K., Demers, A. J., Spreitzer, M. J., and Hauser, C. H. 1995. Managing update conflicts in Bayou, a weakly connected replicated storage system. In Proc. 15th ACM Symposium on Operating Systems Principles (SOSP '95). ACM, New York, NY, 172–182.

[3] Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M. 2011. Conflict-free replicated data types. In Proc. 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS 2011), Lecture Notes in Computer Science, vol. 6976. Springer, Berlin, 386–400.

[4] Gray, J. and Reuter, A. 1992. Transaction Processing: Concepts and Techniques. Morgan Kaufmann, San Francisco.

[5] Lamport, L. 1998. The part-time parliament. ACM Trans. Comput. Syst. 16, 2 (May 1998), 133–169.

[6] Oki, B. M. and Liskov, B. H. 1988. Viewstamped replication: A new primary copy method to support highly-available distributed systems. In Proc. 7th ACM Symposium on Principles of Distributed Computing (PODC '88). ACM, New York, NY, 8–17.

[7] Brewer, E. A. 2000. Towards robust distributed systems (abstract). In Proc. 19th Annual ACM Symposium on Principles of Distributed Computing (PODC 2000). ACM, New York, NY, 7.

[8] Coda: A Highly Available File System for a Distributed Workstation Environment. Mahadev Satyanarayanan, James J. Kistler, Puneet Kumar, Maria E. Okasaki, Ellen H. Siegel, and David C. Steere. IEEE Transactions on Computers 39, 4 (April 1990), 447–459.(1990)

[9] Satyanarayanan, M. 2001. Pervasive computing: Vision and challenges. IEEE Pers. Comm. 8, 4 (Aug. 2001), 10–17.

[10] DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., and Vogels, W. 2007. Dynamo: Amazon's highly available key-value store. In Proc. 21st ACM Symposium on Operating Systems Principles (SOSP '07). ACM, New York, NY, 205–220.

[11] Lakshman, A. and Malik, P. 2010. Cassandra: A decentralized structured storage system. ACM SIGOPS Operating Systems Review 44, 2 (Apr. 2010), 35–40.

[12] Burrows, M. 2006. The Chubby lock service for loosely-coupled distributed systems. In Proc. 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06). USENIX Association, Berkeley, CA, 335–350.

[14] Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M. 2011. A comprehensive study of convergent and commutative replicated data types. Technical Report 7506, INRIA, Rocquencourt, France. Jan. 2011.

[15] Bieniusa, A., Zawirski, M., Preguiça, N., Shapiro, M., Baquero, C., Balegas, V., and Duarte, S. 2012. An optimized conflict-free replicated set. In Proc. European Conference on Computer Systems (EuroSys).

[16] Roh, H. G., Jeon, M., Kim, J. S., and Lee, J. 2011. Replicated abstract data types: Building blocks for collaborative applications. J. Parallel Distrib. Comput. 71, 3 (Mar. 2011), 354–368.

[17]

[18] Ahamad, M., Neiger, G., Burns, J. E., Kohli, P., and Hutto, P. W. 1995. Causal memory: Definitions, Implementation, and Programming. Distrib. Comput. 9, 1 (1995), 37–49.

[19] Gray, J. 1978. Notes on database operating systems. In Operating Systems: An Advanced Course, Lecture Notes in Computer Science, vol. 60. Springer, Berlin, 393–481.

[20] Lamport, L. 2001. Paxos made simple. ACM SIGACT News 32, 4 (Dec. 2001), 51–58.

[21] Lamport, L. 1986. On interprocess communication. Distrib. Comput. 1, 2 (1986), 77–101.

[22] Birman, K. P. and Joseph, T. A. 1987. Reliable communication in the presence of failures. ACM Trans. Comput. Syst. 5, 1 (Feb. 1987), 47–76.

[23] Lamport, L. 1979. How to make a multiprocessor computer that correctly executes multiprocess programs. IEEE Trans. Comput. C-28, 9 (Sept. 1979), 690–691.

[24] Ladin, R., Liskov, B., Shrira, L., and Ghemawat, S. 1992. Providing high availability using lazy replication. ACM Trans. Comput. Syst. 10, 4 (Nov. 1992), 360–391.

[25] Fielding, R., Gettys, J., Mogul, J., Frystyk, H., Masinter, L., Leach, P., and Berners-Lee, T. 1999. Hypertext Transfer Protocol. HTTP/1.1. RFC 2616. Internet Engineering Task Force.

[26] Fielding, R. T. and Taylor, R. N. 2002. Principled design of the modern web architecture. ACM Trans. Internet Technol. 2, 2 (May 2002), 115–150.

[27] Fielding, R. T. 2000. Architectural styles and the design of network-based software architectures. Ph.D. Dissertation. University of California, Irvine.

[28] Nuno Preguiça, João Marques, Marc Shapiro, and Macedonio Letia. A Commutative Replicated Data Type for Cooperative Editing. (2009)

[29] Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M. 2011. Conflict-free replicated data types. Technical Report 7686, INRIA, Rocquencourt, France. July 2011. [3]

[30] Baquero, C., Almeida, P. S., and Shoker, A. 2017. Pure operation-based replicated data types. arXiv:1710.04469.

[31] Birman, K. P., Schiper, A., and Stephenson, P. 1991. Lightweight causal and atomic group multicast. ACM Trans. Comput. Syst. 9, 3 (Aug. 1991), 272–314.

[32] Fidge, C. J. 1988. Timestamps in message-passing systems that preserve the partial ordering. In Proc. 11th Australian Computer Science Conference, vol. 10. 56–66.

[33] Mattern, F. 1989. Virtual time and global states of distributed systems. In Proc. International Workshop on Parallel and Distributed Algorithms. North-Holland, Amsterdam, 215–226.

[34] Schwarz, R. and Mattern, F. 1994. Detecting causal relationships in distributed computations: In search of the Holy Grail. Distrib. Comput. 7, 3 (1994), 149–174.

[35] Fischer, M. J., Lynch, N. A., and Paterson, M. S. 1985. Impossibility of distributed consensus with one faulty process. J. ACM 32, 2 (Apr. 1985), 374–382.

[36] Elnozahy, E. N., Alvisi, L., Wang, Y. M., and Johnson, D. B. 2002. A survey of rollback-recovery protocols in message-passing systems. ACM Comput. Surv. 34, 3 (Sept. 2002), 375–408.

[37] Lamport, L., Shostak, R., and Pease, M. 1982. The Byzantine generals problem. ACM Trans. Program. Lang. Syst. 4, 3 (July 1982), 382–401.

[38] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. The Google File System (2003)

[39] Rosenblum, M. and Ousterhout, J. K. 1992. The design and implementation of a log-structured file system. ACM Trans. Comput. Syst. 10, 1 (Feb. 1992), 26–52.

[40] Herlihy, M. 1991. Wait-free synchronization. ACM Trans. Program. Lang. Syst. 13, 1 (Jan. 1991), 124–149.

[41] Attiya, H. and Welch, J. L. 2004. Distributed Computing: Fundamentals, Simulations, and Advanced Topics, 2nd ed. Wiley-Interscience, Hoboken, NJ.

[42] Androutsellis-Theotokis, S. and Spinellis, D. 2004. A survey of peer-to-peer content distribution technologies. ACM Comput. Surv. 36, 4 (Dec. 2004), 335–371.

[43] Gilbert, S. and Lynch, N. 2002. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. SIGACT News 33, 2 (June 2002), 51–59.

[44] Hadzilacos, V. and Toueg, S. 1993. Fault-tolerant broadcasts and related problems. In Mullender, S. (Ed.), Distributed Systems, 2nd ed. ACM Press/Addison-Wesley, New York, NY, 97–145.

[45] Ellis, C. A. and Gibbs, S. J. 1989. Concurrency control in groupware systems. In Proc. 1989 ACM SIGMOD International Conference on Management of Data. ACM, New York, NY, 399–407.

[46] Sun, C., Jia, X., Zhang, Y., Yang, Y., and Chen, D. 1998. Achieving convergence, causality preservation, and intention preservation in real-time cooperative editing systems. ACM Trans. Comput.-Hum. Interact. 5, 1 (Mar. 1998), 63–108.

[47] Imine, A., Molli, P., Oster, G., and Rusinowitch, M. 2003. Proving correctness of transformation functions in real-time groupware. In Proc. 8th European Conference on Computer-Supported Cooperative Work (ECSCW 2003). Kluwer Academic Publishers, Norwell, MA, 277–293.

[48] Golding, R. A. 1992. A weak-consistency architecture for distributed information services. Comput. Syst. 5, 4 (Fall 1992), 379–405.

[49] Saito, Y. and Shapiro, M. 2005. Optimistic replication. ACM Comput. Surv. 37, 1 (Mar. 2005), 42–81.

[50] Vogels, W. 2008. Eventually consistent. ACM Queue 6, 6 (Oct. 2008), 14–19.

[51] Bernstein, P. A. and Newcomer, E. 2009. Principles of Transaction Processing, 2nd ed. Morgan Kaufmann, Burlington, MA.

[52] Nichols, D. A., Curtis, P., Dixon, M., and Lamping, J. 1995. High-latency, low-bandwidth windowing in the Jupiter collaboration system. In Proc. 8th Annual ACM Symposium on User Interface Software and Technology (UIST '95). ACM, New York, NY, 111–120.

[53] Cai, M., Frank, M., Chen, J., and Szekely, P. 2003. MAAN: A multi-attribute addressable network for grid information services. In Proc. 4th International Workshop on Grid Computing. IEEE, 184–191.

[54] Flinn, J., Park, S., and Satyanarayanan, M. 2002. Balancing performance, energy, and quality in pervasive computing. In Proc. 22nd International Conference on Distributed Computing Systems (ICDCS 2002). IEEE, 217–226.

[55] Klophaus, R. 2010. Riak Core: Building distributed applications without shared state. In Proc. ACM SIGPLAN Erlang Workshop. ACM, New York, NY, 14–17.

[56] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. 2016. Borg, Omega, and Kubernetes. Commun. ACM 59, 5 (May 2016), 50–57.